

LangChain — Origin, Architecture & Execution Flow

Complete conceptual notes: why LangChain came into existence, how its abstractions (Runnable, BaseLLM, BaseChatModel) work, why provider integrations (HuggingFace vs OpenAI) differ, and the full execution flow from prompt to final output.

1. Before LangChain — Problems with the Manual Approach

Before using a framework like LangChain, a developer had to handle almost everything manually in Python to build an LLM-based application — especially a RAG pipeline. Sending raw text to a model's API, getting the response back as a plain string, manually parsing that string, manually merging system instructions with the user's question into a single prompt string, manually inserting documents returned by a retriever into the context, and manually tracking multi-turn conversation history — all of this was the developer's own responsibility.

The biggest problem was that there was no common or standard interface. If the model provider had to be changed (say, from OpenAI to HuggingFace, or from HuggingFace to Anthropic), the integration code had to be rewritten almost from scratch, because each provider had its own request/response format. Prompting, model-calling, and output-parsing were three independent, tightly-coupled pieces of code, and there was no consistent way to connect (chain) them together.

2. Base LLM — The Old String-in, String-out Style

In the beginning, LLMs were treated as “text completion” models — the input was a plain string, and the output was a plain string too. There was no structured concept of which part was the system instruction, which part was the human message, and which part was the model's own previous response — everything got mixed together into a single string (something like “System: ...\nUser: ...\nAssistant:”). This was purely convention-based — the model had no built-in or guaranteed understanding of this format. As a result, clearly separating which message belonged to whom, or building a proper multi-turn conversation, was fragile and error-prone in this style.

3. LangChain's Solution — the Runnable Abstraction

To solve this problem, LangChain created a “master class” called **Runnable**. Runnable is not a model itself — it is a universal contract (a “universal plug”). The rule is simple: any component that wants to be part of the LangChain ecosystem — whether it's a prompt template, a model, or an output parser — must derive from the Runnable class.

This gives two major benefits:

- Every component gets a common method — **invoke()** — so all of them can be called consistently, whether it's a prompt, a model, or a parser.
- Every component can easily be joined into a chain using the pipe operator (`|`) — for example, `prompt | model | output_parser`.

4. Two Interfaces — BaseLLM and BaseChatModel

From Runnable, LangChain created two separate abstract classes, because there were two kinds of models in the market:

- **BaseLLM** (the old style) — takes a single string as input, returns a string as output.
- **BaseChatModel** (the new style) — takes a list of messages (System, Human, AI messages) as input, returns an AIMessage object as output.

Both are subclasses of Runnable, so both automatically get `invoke()` — without writing it themselves.

5. What “Interface” Really Means — the Abstract Method Contract

Python doesn't have an explicit “interface” keyword like Java, so LangChain uses Abstract Base Classes (ABC) instead. Both BaseLLM and BaseChatModel are abstract classes with one method marked as `@abstractmethod`:

```
BaseLLM          -> _call(prompt: str, ...) -> str
BaseChatModel    -> _generate(messages: List[BaseMessage], ...) -> ChatResult
```

This means: any provider (OpenAI, HuggingFace, Anthropic) that wants to integrate its model into LangChain must choose one of these two classes and implement that specific method — otherwise Python raises an error the moment the object is instantiated. This is what “following the interface” means. Importantly, providers do not need to write `invoke()` themselves — LangChain has already written `invoke()` in such a way that it internally calls `_call()` or `_generate()` for you.

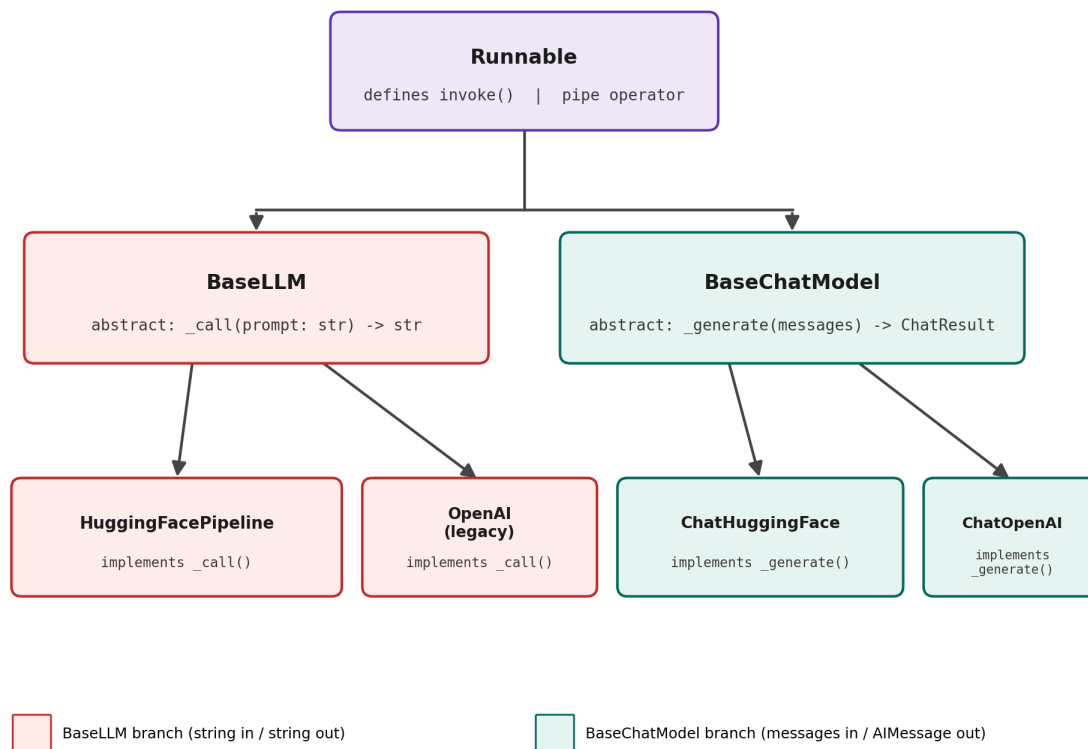


Figure 1: Runnable is the common root. BaseLLM and BaseChatModel are two sibling abstract classes derived from it, each defining a different abstract method (`_call` vs `_generate`). Concrete provider classes implement one branch or the other.

6. Structured Messages — SystemMessage, HumanMessage, AIMessage

The BaseChatModel interface only delivers its full benefit when the input is given as structured messages. LangChain provides three main message types:

- **SystemMessage** — instructions/persona for the model.
- **HumanMessage** — the user's input.
- **AIMessage** — the model's response, or a previous AI turn in the conversation history.

These messages are passed to a BaseChatModel as a list, and the model always knows clearly which role belongs to which message — there is no ambiguity, unlike the Base LLM style.

7. Provider Integration — Every Provider Fulfils Its Own Contract

Whenever a provider (OpenAI, HuggingFace, Anthropic) adds its model to LangChain, it decides whether its underlying API is of the old completion-style, or the new chat-style, and implements BaseLLM or BaseChatModel accordingly. This choice directly affects what the user sees: if the provider implemented BaseChatModel, the user gets structured input/output; if it only implemented BaseLLM, the input gets flattened into a plain string and the output remains a plain string too.

8. The HuggingFace Case — Two-Step Wrapping

HuggingFace's raw text-generation pipeline (`HuggingFacePipeline`) is fundamentally an old-style completion tool — it implements the **BaseLLM** interface. To make it chat-capable, LangChain provides a separate wrapper class: `ChatHuggingFace`. This wrapper class implements the **BaseChatModel** interface, and holds a `HuggingFacePipeline` (base LLM) object inside itself — there is no new model inside it, it's the same old one, just wrapped one layer above.

When `ChatHuggingFace._generate()` is called, it performs these steps internally:

- 1 Takes the structured messages coming from outside.
- 2 Uses the model's tokenizer's `apply_chat_template()` function to convert them into the exact prompt-string format the model was trained on (something like `<|system|>...<|user|>...`).
- 3 Passes this formatted string to the `HuggingFacePipeline` it holds inside.
- 4 Takes the raw text the pipeline returns and packs it back into an `AIMessage` object, so the `BaseChatModel` contract is satisfied.

9. The ChatOpenAI Case — Why the Wrapping Isn't Visible

With `ChatOpenAI`, the user doesn't need to do any two-step wrapping — simply writing `ChatOpenAI(model="...")` gives a ready-to-use chat object. The reason: OpenAI's own underlying API is already message-based by nature (the Chat Completions endpoint accepts role-based messages) — so the `ChatOpenAI` class can directly implement the **BaseChatModel** interface, without wrapping any old string-based class inside it.

(OpenAI also has a separate legacy `OpenAI` class that implements **BaseLLM** for the old text-completion endpoint, but `ChatOpenAI` has nothing to do with it — the two are independent implementations.)

So this difference is not really about “HuggingFace vs OpenAI” — it's about whether the underlying provider API is itself message-based or not:

- Where the underlying API only accepts raw text (like the raw HF pipeline) — LangChain needs a translation layer (a wrapper).
- Where the underlying API is already structured (like OpenAI's Chat Completions) — a direct `BaseChatModel` implementation is enough, and no wrapping is visible.

10. How `invoke()` Works Internally

- 1 `invoke()` is called — this method comes from `Runnable`, but `BaseChatModel` has overridden its own `invoke()` to handle extra chat-specific logic.
- 2 Internally, this `invoke()` automatically calls `_generate()` (if it's a chat model) or `_call()` (if it's a base LLM).
- 3 If the model is a wrapper (like `ChatHuggingFace`), `_generate()` translates the messages and then calls the base LLM held inside it.
- 4 The raw response comes back from the provider.
- 5 The response is converted back into LangChain's standard representation (`AIMessage`, or a plain string in the `BaseLLM` case).
- 6 This standard representation is returned to the user through `invoke()`.

11. Execution Flow — How the Full Chain Runs

The flow below differs from a generic linear diagram in one important way: **the data going from the prompt to the model takes a different shape depending on whether the model is a `BaseLLM` or a `BaseChatModel`** (a flat string vs. a message list) — this is not a fixed universal step, it depends on the model's interface type.

Suppose the chain looks like this: `chain = prompt | model | output_parser`, and we call `chain.invoke(input)`.

- STEP 1 `ChatPromptTemplate.invoke(input)`
Placeholders (`{question}/{context}`) are filled in.
Result: a `ChatPromptValue` object, holding messages (`SystemMessage`, `HumanMessage`) as a list.
- STEP 2 The `ChatPromptValue` is piped into the next `Runnable` (the model).
LangChain checks: is the next `Runnable` a `BaseLLM` or a `BaseChatModel`?
- > If it is a `BaseLLM`:
 `.to_string()` is called
 All messages are collapsed into a FLAT STRING
 ("System: ...\nHuman: ...")
- > If it is a `BaseChatModel`:
 `.to_messages()` is called
 Messages remain a LIST, no flattening happens
- STEP 3 The model's `invoke()` is triggered.
Internally this calls either `_call(prompt: str)` [`BaseLLM`]
or `_generate(messages)` [`BaseChatModel`]
- STEP 4 If the model is a wrapper (e.g. `ChatHuggingFace`):
 `_generate()` translates the messages into the provider-specific format (`tokenizer.apply_chat_template()`), then calls the base model (`HuggingFacePipeline`) held inside it — the same kind of text-generation call that would happen for a `BaseLLM` in Step 3.
- STEP 5 The underlying model/API performs the actual text generation.
- STEP 6 The response is wrapped back into LangChain's standard type:

```

came from BaseLLM      -> plain string
came from BaseChatModel -> AIMessage object

STEP 7 This output is piped into the next Runnable (StrOutputParser):
      got an AIMessage -> extracts .content
      got a string      -> passes it through as-is

STEP 8 The final clean string reaches the user
      (the return value of chain.invoke()).

```

12. The Role of the Output Parser

An output parser is also a Runnable. Its job is simple — take whatever came out of the model (whether an AIMessage or a plain string) and convert it for downstream use. StrOutputParser is the simplest parser — it just extracts `.content` (if given an AIMessage) or passes a string through as-is (if it's already a string). There are also structured output parsers (like PydanticOutputParser) that convert raw text/JSON into a defined Python object.

Important point: the output parser itself carries no “prompt cleaning” or “extra intelligence” — it only performs type conversion. Whatever difference exists in the shape of the prompt/output has already been decided in Steps 2–6 above.

13. How This Architecture Fits into a RAG Pipeline

A RAG pipeline has several components — a retriever (which pulls relevant documents from a vector store), a prompt template (which combines the retrieved context with the user's question), a model (which generates the answer), and an output parser (which produces the final clean answer). Because every component is a Runnable, all of them can be joined into a single pipe-chain:

```

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | model
    | output_parser
)
response = rag_chain.invoke("the user's question")

```

The retriever is itself a Runnable (it returns documents via `invoke()`). So the entire RAG pipeline — from retrieval to the final answer — runs through one consistent `invoke()`-based interface, no matter which provider's model sits underneath.

This is LangChain's real value: once this architecture (Runnable → BaseLLM/BaseChatModel → provider implementation → invoke-flow) is understood, switching to a new model provider becomes as simple as changing one line — the rest of the pipeline (prompt, retriever, parser) stays exactly the same.

14. Code Reference — Short Conceptual Examples

14.1 — Base LLM style (string in / string out)

```
from langchain_core.language_models.llms import LLM

class MyBaseLLM(LLM):
    @property
    def _llm_type(self) -> str:
        return "my_base_llm"

    def _call(self, prompt: str, stop=None, **kwargs) -> str:
        return call_some_string_api(prompt)  # string in, string out
```

14.2 — Chat Model style (messages in / AIMessage out)

```
from langchain_core.language_models.chat_models import BaseChatModel
from langchain_core.messages import AIMessage
from langchain_core.outputs import ChatResult, ChatGeneration

class MyChatModel(BaseChatModel):
    @property
    def _llm_type(self) -> str:
        return "my_chat_model"

    def _generate(self, messages, stop=None, **kwargs) -> ChatResult:
        text = call_some_chat_api(messages)  # messages list in
        msg = AIMessage(content=text)
        return ChatResult(generations=[ChatGeneration(message=msg)])
```

14.3 — ChatPromptTemplate

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Context:\n{context}"),
    ("user", "{question}"),
])
```

14.4 — HuggingFace: base LLM + wrapper

```
from langchain_huggingface import HuggingFacePipeline, ChatHuggingFace

base_llm = HuggingFacePipeline.from_model_id(
    model_id="Qwen/Qwen2.5-1.5B-Instruct",
    task="text-generation",
    pipeline_kwargs={"max_new_tokens": 256, "return_full_text": False},
)
chat_model = ChatHuggingFace(llm=base_llm)  # wrapper -> becomes a BaseChatModel
```

14.5 — ChatOpenAI: direct chat model, no separate wrapping

```
from langchain_openai import ChatOpenAI

chat_model = ChatOpenAI(model="gpt-4o-mini")  # ready to use directly
# (because OpenAI's Chat Completions API is already message-based)
```

14.6 — Output parser + the full chain

```
from langchain_core.output_parsers import StrOutputParser
```

```
chain = prompt | chat_model | StrOutputParser()
response = chain.invoke({"question": "...", "context": "..."})
print(response)    # -> just the clean answer, not the AIMessage or the prompt
```

End of notes — LangChain background, abstractions, provider integration, and execution flow.